Actividad 5. Regresión Logística

Primero se importó la base drugs.csv y se analizaron sus columnas. En la misma hay 200 pacientes y seis variables originales: edad, sexo, presión arterial, colesterol, Na_to_K y medicamento. La base incluye cinco medicamentos diferentes: A, B, C, X y Y. Los tres primeros son de proveedor nacional, mientras que X y Y son de proveedor extranjero.

El enunciado menciona como variables: edad, sexo, presión arterial y colesterol, pero la tabla que se proporciona también contiene Na_to_K (decidí dejarla por motivos de que fuera una omisión accidental).

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn import metrics
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

from sklearn import preprocessing

import os
os.chdir("D:/gus6e/Documents")
```

```
col_names = ["Age", "Sex", "BP", "Cholesterol", "Na_to_K", "Drug"]
pima = pd.read_csv("drugs.csv", header = None, names = col_names)
df = pima.iloc[1:,:]
df
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
1	23	F	HIGH	HIGH	25.355	drugY
2	47	M	LOW	HIGH	13.093	drugC
3	47	M	LOW	HIGH	10.114	drugC
4	28	F	NORMAL	HIGH	7.798	drugX
5	61	F	LOW	HIGH	18.043	drugY
...	...	...	...	...	...	...
196	56	F	LOW	HIGH	11.567	drugC
197	16	M	LOW	HIGH	12.006	drugC

Como algunos datos estaban escritos con palabras, utilicé el código proporcionado en las diapositivas de la actividad “processing.LabelEncoder”, tal como se pide, para convertirlos en números. En la variable sexo, F quedó representado por 0 y M por 1. Para la presión arterial, HIGH se convirtió en 0, LOW en 1 y NORMAL en 2. En colesterol, HIGH se convirtió en 0 y NORMAL en 1. Esta conversión era necesaria para utilizar esas características en la regresión logística que aplicamos.

```
pima = preprocessing.LabelEncoder()
df["Sex"] = pima.fit_transform(df["Sex"])
df["BP"] = pima.fit_transform(df["BP"])
df["Cholesterol"] = pima.fit_transform(df["Cholesterol"])

df["label"] = df["Drug"].map({"drugA":0, "drugB":0, "drugC":0, "drugX":1, "drugY":1})
df.head()
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug	label
1	23	0	0	0	25.355	drugY	1
2	47	1	1	0	13.093	drugC	0
3	47	1	1	0	10.114	drugC	0
4	28	0	2	0	7.798	drugX	1
5	61	0	1	0	18.043	drugY	1

Después se transformó el medicamento en la variable que realmente se quiere pronosticar: asignando el valor 0 a los medicamentos nacionales A, B y C, y el valor 1 a los extranjeros X y Y. Así ya no intentamos identificar uno de los cinco nombres de medicamentos, sino únicamente el origen de su proveedor.

```
# Selección de las características
feature_cols = ["Age", "Sex", "BP", "Cholesterol", "Na_to_K"]
x = df[feature_cols]
y = df.label
```

Una vez preparadas las variables, se divide la base de la misma manera que en la lección previa a la actividad. Utilizando el 80 % para entrenar el modelo y el 20 % para comprobar sus resultados.

Esta separación nos ayuda a evaluar la capacidad predictiva del modelo, porque los resultados se revisan con observaciones distintas de las que se usaron para ajustarlo.

```
# Selección de las características
feature_cols = ["Age", "Sex", "BP", "Cholesterol", "Na_to_K"]
x = df[feature_cols]
y = df.label
```

```
# Creación de las bases de entrenamiento de prueba
x_train, x_test,y_train,y_test = train_test_split(x,y,test_size = 0.2, random_state = 1)
```

```
# sag: Stochastic Average Gradient
# newton-cg: Newton
```

Posteriormente, se aplicaron los cinco solucionadores de regresión logística que se revisaron en clase: sag, newton-cg, liblinear, saga y lbfgs. Cambian en la forma en que buscan los parámetros del modelo. Con cada se observó su precisión global y el reporte de clasificación, sin cambiar la base de entrenamiento ni la de prueba.

El solucionador sag alcanzó una precisión global del 90 %. Aunque logró identificar correctamente todos los casos extranjeros de la muestra de prueba, solamente identificó 6 de los 10 casos nacionales. Por lo tanto, su resultado general fue bueno, pero presentó dificultades para distinguir una parte de los medicamentos nacionales.

Regresión logísitca con solucionador (Solver): sag

```
model = LogisticRegression(solver="sag")
clf = model.fit(x_train,y_train)
```

```
# Predicción de etiquetas de clase sobre datos de prueba
y_pred = model.predict(x_test)

# Impresión de coeficientes de la regresión de puntajes
print("Intercepto (Beta 0)",clf.intercept_)
print("Pesos de cada variable (Beta1, Beta2, ... , Beta5)",clf.coef_)
```

```
Intercepto (Beta 0) [-0.31195197]
Pesos de cada variable (Beta1, Beta2, ... , Beta5) [[-0.03748102 -0.16242261  1.03069788  0.20122708  0.16022598]]
```

```
# Evaluación de la precisión del modelo
score = model.score(x_test, y_test)
print("Score de precisión: ", score)
```

```
Score de precisión:  0.9
```

```
# Reporte de clasificación
print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      0.60      0.75        10
           1       0.88      1.00      0.94        30

    accuracy                           0.90        40
   macro avg       0.94      0.80      0.84        40
weighted avg       0.91      0.90      0.89        40
```

Después se probó newton-cg, que alcanzó una precisión del 97.5 %. Este modelo clasificó correctamente a 39 de los 40 pacientes de prueba. Identificó los 30 casos extranjeros y 9 de los 10 nacionales. Esto representa una mejora importante respecto a sag, especialmente en la clasificación de los medicamentos nacionales.

Regresión logística con solucionador (Solver): newton-cg

```
model = LogisticRegression(solver = "newton-cg")
clf = model.fit(x_train,y_train)

y_pred = model.predict(x_test)

print("Intercepto (Beta 0):", clf.intercept_)
print("Pesos (Betas):", clf.coef_)
print("_____________________________________________________________")
score = model.score(x_test,y_test)
print("Precisión global:", score)
print("_____________________________________________________________")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta 0): [-11.15244637]
Pesos (Betas): [[0.00660794 0.1483032  3.33609316 2.07244718 0.6378735 ]]
_____________________________________________________________
Precisión global: 0.975
_____________________________________________________________
              precision    recall  f1-score   support

           0       1.00      0.90      0.95        10
           1       0.97      1.00      0.98        30

    accuracy                           0.97        40
   macro avg       0.98      0.95      0.97        40
weighted avg       0.98      0.97      0.97        40
```

Con liblinear también se obtuvo una precisión global del 97.5 % y el mismo número de aciertos en los dos tipos de proveedor.

Regresión logística con solucionador (Solver): liblinear

```
model = LogisticRegression(solver = "liblinear")
clf = model.fit(x_train,y_train)

y_pred = model.predict(x_test)

print("Intercepto (Beta 0):", clf.intercept_)
print("Pesos (Betas):", clf.coef_)
print("_____________________________________________________________")
score = model.score(x_test,y_test)
print("Precisión global:", score)
print("_____________________________________________________________")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta 0): [-3.34051275]
Pesos (Betas): [[-0.02834219 -0.26053256  2.06969328  1.09570978  0.27510142]]
_____________________________________________________________
Precisión global: 0.975
_____________________________________________________________
              precision    recall  f1-score   support

           0       1.00      0.90      0.95        10
           1       0.97      1.00      0.98        30

    accuracy                           0.97        40
   macro avg       0.98      0.95      0.97        40
weighted avg       0.98      0.97      0.97        40
```

En cambio, saga alcanzó 87.5 %: reconoció 29 de los 30 casos extranjeros, pero solo 6 de los 10 nacionales.

Regresión logística con solucionador (Solver): saga

```
model = LogisticRegression(solver = "saga")
clf = model.fit(x_train,y_train)

y_pred = model.predict(x_test)

print("Intercepto (Beta 0):", clf.intercept_)
print("Pesos (Betas):", clf.coef_)
print("_____________________________________________________________")
score = model.score(x_test,y_test)
print("Precisión global:", score)
print("_____________________________________________________________")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta 0): [-0.14681563]
Pesos (Betas): [[-0.03375957 -0.09257693  0.66067608  0.10531952  0.15456863]]
_____________________________________________________________
Precisión global: 0.875
_____________________________________________________________
              precision    recall  f1-score   support

           0       0.86      0.60      0.71        10
           1       0.88      0.97      0.92        30

    accuracy                           0.88        40
   macro avg       0.87      0.78      0.81        40
weighted avg       0.87      0.88      0.87        40
```

Por último, se aplica el lbfgs, que logró una precisión global de 97.5 %, igual que newton-cg y liblinear. También clasificó correctamente a 39 de los 40 pacientes de prueba.

Regresión logística con solucionador (Solver): lbfgs

```
model = LogisticRegression(solver = "lbfgs")
clf = model.fit(x_train,y_train)

y_pred = model.predict(x_test)

print("Intercepto (Beta 0):", clf.intercept_)
print("Pesos (Betas):", clf.coef_)
print("_____________________________________________________________")
score = model.score(x_test,y_test)
print("Precisión global:", score)
print("_____________________________________________________________")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta 0): [-11.16791614]
Pesos (Betas): [[0.00667297 0.14894912 3.33863745 2.07461152 0.63862512]]
_____________________________________________________________
Precisión global: 0.975
_____________________________________________________________
              precision    recall  f1-score   support

           0       1.00      0.90      0.95        10
           1       0.97      1.00      0.98        30

    accuracy                           0.97        40
   macro avg       0.98      0.95      0.97        40
weighted avg       0.98      0.97      0.97        40
```

Para esta actividad elige lbfgs. Obtuvo una precisión global del 97.5 % y un buen desempeño al identificar tanto medicamentos nacionales como extranjeros. Además, al ser el último solucionador que probamos en la estructura de la Lección 5, se puede continuar directamente con el cálculo de probabilidades y la gráfica ROC.

```
# Predicción de probabilidades para casos particulares
y_pred_probs = clf.predict_proba(x_test)[:,1]
y_pred_probs[0:5]
```

```
array([0.9898257 , 0.99974436, 0.99355295, 0.67717001, 0.99999695])
```

```
y_test[0:5]
```

```
59     1
41     1
35     1
103    0
185    1
Name: label, dtype: int64
```

```
y_test = y_test.to_numpy()
y_test[0:5]
```

```
array([1, 1, 1, 0, 1])
```

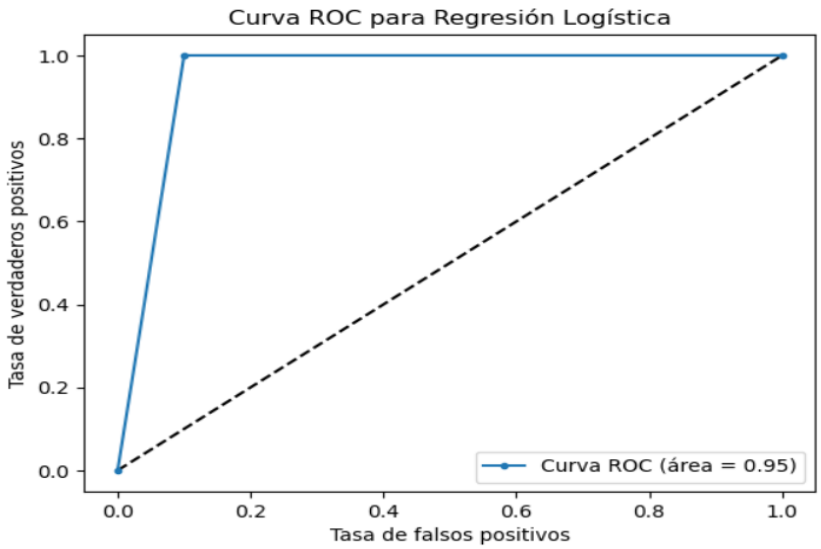
```
import numpy as np
y_pred_probs = np.round(y_pred_probs,0)
y_pred_probs[0:5]
```

```
array([1., 1., 1., 1., 1.])
```

Después del reporte de clasificación, realizamos la curva ROC utilizando el modelo de regresión logística con el método lbfgs, siguiendo el procedimiento de la Lección 5. En la gráfica obtuvimos un área bajo la curva (AUC) de 0.95, lo que indica que el modelo tiene una muy buena capacidad para distinguir entre los medicamentos nacionales y extranjeros. Además, observamos que alcanza una tasa de verdaderos positivos del 100%, con aproximadamente un 10% de falsos positivos en la parte que . Esto significa que el modelo logra identificar correctamente los casos positivos, aunque todavía puede cometer algunos errores al clasificar los negativos.

```
Área bajo la curva (AUC): 0.9500000000000001
```

```
plt.plot([0,1],[0,1], "k--")
plt.plot(fpr,tpr,marker= ".", label = "Curva ROC (área = %0.2f)" % auc)
plt.xlabel("Tasa de falsos positivos") #False positive rate
plt.ylabel("Tasa de verdaderos positivos") #True positive rate
plt.legend(loc = "lower right")
plt.title("Curva ROC para Regresión Logística")
plt.show()
```



¿Cómo se convirtieron las variables cualitativas a una escala numérica?

Utilizando LabelEncoder para transformar sexo, presión arterial y colesterol en valores numéricos. Después la variable del medicamento en dos categorías: 0 para proveedor nacional y 1 para proveedor extranjero.

¿Qué método de optimización recomendaría?

Recomendaría lbfgs porque obtuvo 97.5 % de precisión global y resultados altos para las dos clases. Sin embargo, newton-cg y liblinear también consiguieron 97.5 % en esta muestra,

¿Qué tan eficaz fue el algoritmo elegido?

Con lbfgs, el modelo clasificó correctamente a 39 de los 40 pacientes de prueba. Los valores de precision, recall y f1-score fueron favorables en las dos categorías, aunque se cometió un error con un medicamento nacional. Además, la curva ROC obtuvo un AUC de 1.00, lo que muestra una separación muy buena entre las clases en estos datos. Aun así, el resultado corresponde solamente a esta muestra y no prueba que el modelo pueda decidir por sí solo qué medicamento debe recibir una persona.